

Тестовое задание

1. Контекст

После путешествия Элли в загадочное место, окруженное скалами, и ее благополучного возвращения домой, рассказ об этом разлетелся по миру. Люди стали предпринимать попытки найти Волшебную Страну. Прознав об этом, Страшила Мудрый понял: если люди смогут свободно посещать его владения, то он хочет хотя бы контролировать этот процесс.

Узнал об этом Страшила совсем недавно, когда Дровосек рассказал ему, что у людей за скалами тоже есть много магических вещей. Таких, как телефоны, интернет, Telegram и др..

И вот, учитывая, что большинство людей в поисках нового волшебства записывают видео в свои блоги, Страшила решил следить за статистикой этих видео, чтобы в случае необходимости он мог своевременно вмешаться.

Подданные Страшилы магическим образом смогли запустить внутренний сервис сбора статистики по видео-креаторам публикующим контент в разных соц сетях.

Сервис хранит две таблицы:

- итоговую статистику по каждому видео (просмотры, лайки, комментарии и т.д.);
- почасовые «снимки» статистики по каждому видео (чтобы отслеживать динамику).

Вам нужно сделать Telegram-бота, который умеет по запросу на естественном языке считать нужные метрики по этим данным. Благодаря вашей работе, Страшила сможет легко и быстро следить за статистикой.

2. Что нужно сделать

1. **Развернуть базу данных (postgresql)**
2. **Загрузить предоставленный JSON-файл** в базу, разложив его на нормальные таблицы. Json файл скачать [тут -> ссылка](#)
3. **Реализовать Telegram-бота**, который:
 - принимает текстовые сообщения от пользователя бота на русском (естественный язык);
 - по этому тексту строит запрос к базе данных;
 - возвращает пользователю ответ - одно число (счётчик, сумму или прирост в зависимости от запроса).
4. **Распознавание естественного языка:**

- Можно использовать любое доступное API. Например, для простоты доступа в РФ и бесплатных запросов можно воспользоваться gigachat, но выбор за вами.
5. **Важно:** вам нужно продумать и написать такой промпт/описание схемы данных, чтобы модель сама понимала, из каких таблиц что брать и как считать.
-

3. Данные

В архиве — один JSON-файл, в котором лежит массив объектов `videos`.
Каждый объект — это одно видео с вложенным списком почасовых снимков.

Таблица `videos` (итоговая статистика по ролику)

- `id` — идентификатор видео;
- `creator_id` — идентификатор креатора;
- `video_created_at` — дата и время публикации видео;
- `views_count` — финальное количество просмотров;
- `likes_count` — финальное количество лайков;
- `comments_count` — финальное количество комментариев;
- `reports_count` — финальное количество жалоб;
- служебные поля `created_at`, `updated_at`.

Таблица `video_snapshots` (почасовые замеры по ролику)

Каждый снимок относится к одному видео и содержит:

- `id` — идентификатор снимка;
- `video_id` — ссылка на соответствующее видео;
- текущие значения: `views_count`, `likes_count`, `comments_count`, `reports_count` на момент замера;
- приращения: `delta_views_count`, `delta_likes_count`, `delta_comments_count`, `delta_reports_count` — насколько изменилось

значение с прошлого замера;

- `created_at` — время замера (раз в час);
 - `updated_at` — служебное поле.
-

4. Требования к боту

Бот должен уметь отвечать на вопросы вида (примеры):

- «Сколько всего видео есть в системе?»
- «Сколько видео у креатора с id `...` вышло с 1 ноября 2025 по 5 ноября 2025 включительно?»
- «Сколько видео набрало больше 100 000 просмотров за всё время?»
- «На сколько просмотров в сумме выросли все видео 28 ноября 2025?»
- «Сколько разных видео получали новые просмотры 27 ноября 2025?»

Характер запроса:

- вопросы задаются **на русском, естественным языком**;
- даты могут быть в виде: «28 ноября 2025», «с 1 по 5 ноября 2025» и т.п.;
- в ответе от бота ожидается **одно число**

Мы будем автоматически прогонять по вашему боту набор своих запросов (в духе примеров выше) и сравнивать ответы с эталонными.

5. Требования к реализации

Вы можете писать задание на любом языке, используя любые доступные технологии и фреймворки, но есть некоторый пулл инструментов, за которые можно получить дополнительные баллы.

Технологии на доп. баллы:

- СУБД — PostgreSQL;
- Docker compose для запуска проекта локально одной командой;
- Использование асинхронного стека (для написания бота на python подойдет например aiogram, также асинхронными должны быть запросы к бд и api);

Внутренняя логика:

- **один запрос к боту → один числовой ответ**
Хранить контекст диалога не нужно (т.е. от вас не требуется разворачивать redis или устанавливать иной memory storage в диспетчер бота)
-

6. Как мы будем проверять

После того как вы развернули бота и он доступен в Telegram, вам нужно запустить проверку через нашего служебного бота **@PPS_Check_bot**.

1. Откройте чат с **@PPS_Check_bot**.

■ Отправьте ему команду в формате:

`/check @yourbotnickname https://github.com/yourrepo Фамилия`

2. Где:

- `@yourbotnickname` — никнейм вашего Telegram-бота, которого нужно проверить;
- `https://github.com/yourrepo` — ссылка на репозиторий с исходным кодом. Убедитесь, что репозиторий публичный;
- `Фамилия` — ваша фамилия;

3. Наш служебный бот автоматически:

- пройдёт по набору тестовых запросов к вашему боту;
- дожждётся ответов;
- сравнит ответы с эталонными значениями.

Важно: ваш бот на момент проверки **должен быть запущен и доступен**, иначе тесты просто не пройдут.

Результат проверки:

- если все тестовые запросы прошли корректно — служебный бот напишет вам, что проверка успешна;
- если в каком-то из промптов будет неправильный расчёт — вы получите сообщение:
 - с указанием, на каком именно запросе произошла ошибка;
 - и с фактом, что ответ не совпал с ожидаемым.

После исправления вы можете снова запустить проверку тем же `/check` (количество попыток не ограничено).

7. Что нужно подготовить

К моменту запуска команды `/check` у вас должно быть:

1. **Работающий Telegram-бот**
 2. **Публичный репозиторий с кодом** (GitHub / GitLab / Bitbucket), ссылка на который передаётся в `/check`. В репозитории должны быть:
 - исходный код бота и всего backend-сервиса;
 - SQL-миграции или скрипты для создания таблиц;
 - скрипт или инструкция по загрузке предоставленного JSON-файла в базу;
 - `README`, в котором описано:
 - как запустить проект локально;
 - инструкция по получению чувствительных данных, таких как токен бота или ключ для нейронной сети (если используете `.env` файл, то создайте и `.env_example` в котором будут названия переменных без чувствительных данных);
 - краткое описание архитектуры и подхода к преобразованию текстовых запросов в обращения к БД (SQL / код);
 - если используется LLM — как именно вы описываете схему данных и какой промпт применяете.
-

8. Как будет оцениваться задание

За тестовое задание можно получить максимум 30 баллов, баллы ставятся за следующее:

1. Чистота кода (оценивается субъективно проверяющим) – 0-2 балла
2. Правильность архитектурных решений (оценивается объективно проверяющим) – 0-2 балла
3. Описание проекта в README соответствует требованиям – 0-2 балла
4. Тесты в служебном боте `@PPS_Check_bot` – 1.5 балла за каждый пройденный тест (всего 8 тестов).
5. Использована PostgreSQL - 4 балла
6. Используется асинхронный стек - 3 балла

7. Проект может быть запущен локально одной командой с помощью docker compose – 5 баллов.

Предполагается, что после запуска проекта база данных будет самостоятельно проинициализирована и заполнена данными с помощью миграций или скрипта. Все зависимости также будут установлены с помощью инструкций из Dockerfile.

Из дополнительных действий в данном пункте возможны:

- Заполнения данными .env файла (если у вас нет .env-example, то укажите в readme какие переменные нужно добавлять. Не забудьте прописать в readme инструкцию для получения этих данных)
- Добавления json с данными в определенную вами директорию (указать в readme), если вы решили не заливать его в репозиторий
- Инструкцию по установке докера прописывать не нужно.

В случае вопросов — пишите в тг @byter1